



UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN
HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 8

NOMBRE COMPLETO: Uriarte Ortiz Enrique Yahir

N° de Cuenta: 318234757

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: Martes 8 de Octubre del 2024.

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas.

- 1) Agregar su dado de 10 caras y editar sus normales para que las caras del dado sean iluminadas correctamente.

Para comenzar con esta actividad primero declare las texturas que utilizare para el dado, las mismas de la practica anterior.

```
Texture AgaveTexture;  
Texture elefanteTexture;  
Texture pulpoTexture;  
Texture mariquitaTexture;  
Texture vacaTexture;  
Texture perico_cerdoTexture;
```

Fig. 1. Definición de texturas para las caras del dado.

Después ajusté las texturas de cada cara, ya que cometí el error en la práctica anterior que en todas las caras las texturas estaban en espejo, para solucionar esto solo invertí las posiciones de los vértices 1 y 3 en cada declaración de cada cara, además los valores de nx, ny y nz los ajuste de acuerdo al cuadrante donde se encontraba cada cara, de este modo la cuando el spotlight iluminaba la pirámide en cada cara se reflejaba correctamente, además para las caras de la punta superior, el reflejo

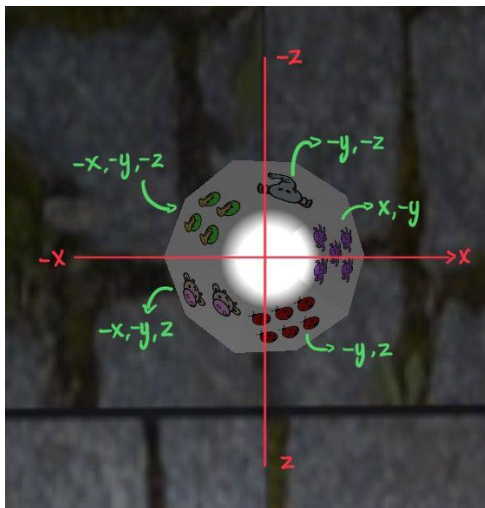


Fig. 2. Análisis de valores para cada cara.

```
void Caras_Elef() {  
    unsigned int cubo_indices[] = {0,1,3, 1,2,3,/*Cara 1*/ 4,5,7, 5,6,7/*Cara 9*/};  
    GLfloat cubo_vertices[] = {  
        //Cara 1  
        1.00f, 0.00f, 3.08f, 0.75f, 0.30f, 0.00f,-1.00f,-1.00f, //A - 0  
        -1.00f,-0.50f, 3.08f, 0.50f, 0.20f, 0.00f,-1.00f,-1.00f, //B - 1  
        -2.62f, 0.00f, 1.90f, 0.25f, 0.30f, 0.00f,-1.00f,-1.00f, //C - 2  
        0.00f, 4.00f, 0.00f, 0.50f, 1.00f, 0.00f,-1.00f,-1.00f, //K - 3  
        //Cara 9  
        2.62f,-0.50f,-1.90f, 0.75f, 0.70f, -1.00f, 1.00f, 0.00f, //H - 4  
        3.24f, 0.00f, 0.00f, 0.50f, 0.80f, -1.00f, 1.00f, 0.00f, //I - 5  
        2.62f,-0.50f, 1.90f, 0.25f, 0.70f, -1.00f, 1.00f, 0.00f, //J - 6  
        0.00f,-4.50f, 0.00f, 0.50f, 0.00f, -1.00f, 1.00f, 0.00f, //L - 7  
    };  
    Mesh* dado = new Mesh();  
    dado->CreateMesh(cubo_vertices, cubo_indices, 64, 12);  
    meshList.push_back(dado);  
}
```

Fig. 3. Ajuste de texturas elefante para las caras del dado.

```

void Caras_Pulpo() {
    unsigned int cubo_indices[] = {0,1,3, 2,1,3,/*Cara 2*/ 4,5,7, 5,6,7/*Cara 7*/};
    GLfloat cubo_vertices[] = {
        //Cara 2
        -2.62f, 0.00f, 1.90f, 0.87f, 0.35f, 1.00f, -1.00f, 0.00f, //C - 0
        -3.24f, -0.50f, 0.00f, 0.50f, 0.25f, 1.00f, -1.00f, 0.00f, //D - 1
        -2.62f, 0.00f, -1.90f, 0.13f, 0.35f, 1.00f, -1.00f, 0.00f, //E - 2
        0.00f, 4.00f, 0.00f, 0.50f, 1.00f, 1.00f, -1.00f, 0.00f, //K - 3
        //Cara 7
        -3.24f, -0.50f, 0.00f, 0.87f, 0.65f, 1.00f, 1.00f, 1.00f, //D - 4
        -2.62f, 0.00f, -1.90f, 0.50f, 0.75f, 1.00f, 1.00f, 1.00f, //E - 5
        -1.00f, -0.50f, -3.08f, 0.13f, 0.65f, 1.00f, 1.00f, 1.00f, //F - 6
        0.00f, -4.50f, 0.00f, 0.50f, 0.00f, 1.00f, 1.00f, 1.00f, //L - 7
    };
    Mesh* dado = new Mesh();
    dado->CreateMesh(cubo_vertices, cubo_indices, 64, 12);
    meshList.push_back(dado);}

```

Fig. 4. Ajuste de texturas pulpo para las caras del dado.

```

void Caras_Mariq() {
    unsigned int cubo_indices[] = {0,1,3, 2,1,3,/*Cara 3*/ 4,5,7, 5,6,7/*Cara 10 */};
    GLfloat cubo_vertices[] = {
        //Cara 3
        -2.62f, 0.00f, -1.90f, 0.90f, 0.35f, 0.00f, -1.00f, 1.00f, //E - 0
        -1.00f, -0.50f, -3.08f, 0.50f, 0.25f, 0.00f, -1.00f, 1.00f, //F - 1
        1.00f, 0.00f, -3.08f, 0.10f, 0.35f, 0.00f, -1.00f, 1.00f, //G - 2
        0.00f, 4.00f, 0.00f, 0.50f, 1.00f, 0.00f, -1.00f, 1.00f, //K - 3
        //Cara 10
        2.62f, -0.50f, 1.90f, 0.90f, 0.65f, 0.00f, 1.00f, -1.00f, //J - 4
        1.00f, 0.00f, 3.08f, 0.50f, 0.75f, 0.00f, 1.00f, -1.00f, //A - 5
        -1.00f, -0.50f, 3.08f, 0.10f, 0.65f, 0.00f, 1.00f, -1.00f, //B - 6
        0.00f, -4.50f, 0.00f, 0.50f, 0.00f, 0.00f, 1.00f, -1.00f, //L - 7
    };
    Mesh* dado = new Mesh();
    dado->CreateMesh(cubo_vertices, cubo_indices, 64, 12);
    meshList.push_back(dado);}

```

Fig. 5. Ajuste de texturas Mariquita para las caras del dado.

```

void Caras_Vaca() {
    unsigned int cubo_indices[] = {0,1,3, 2,1,3,/*Cara 4*/ 4,5,7, 5,6,7/*Cara 8*/};
    GLfloat cubo_vertices[] = {
        //Cara 4
        1.00f, 0.00f, -3.08f, 0.90f, 0.35f, -1.00f, -1.00f, 1.00f, //G - 0
        2.62f, -0.50f, -1.90f, 0.50f, 0.25f, -1.00f, -1.00f, 1.00f, //H - 1
        3.24f, 0.00f, 0.00f, 0.10f, 0.35f, -1.00f, -1.00f, 1.00f, //I - 2
        0.00f, 4.00f, 0.00f, 0.50f, 1.00f, -1.00f, -1.00f, 1.00f, //K - 3
        //Cara 8
        -1.00f, -0.50f, -3.08f, 0.90f, 0.65f, 0.00f, 1.00f, 1.00f, //F - 4
        1.00f, 0.00f, -3.08f, 0.50f, 0.75f, 0.00f, 1.00f, 1.00f, //G - 5
        2.62f, -0.50f, -1.90f, 0.10f, 0.65f, 0.00f, 1.00f, 1.00f, //H - 6
        0.00f, -4.50f, 0.00f, 0.50f, 0.00f, 0.00f, 1.00f, 1.00f, //L - 7
    };
    Mesh* dado = new Mesh();
    dado->CreateMesh(cubo_vertices, cubo_indices, 64, 12);
    meshList.push_back(dado);}

```

Fig. 6. Ajuste de texturas Vaca para las caras del dado.

```

void Caras_PC() {
    unsigned int cubo_indices[] = {0,1,3, 2,1,3,/*Cara 5*/ 4,5,7, 5,6,7/*Cara 6*/};
    GLfloat cubo_vertices[] = {
        //Cara 5
        3.24f, 0.00f, 0.00f, 0.90f, 0.625f, -1.00f, -1.00f, -1.00f, //I - 0
        2.62f, -0.50f, 1.90f, 0.50f, 0.58f, -1.00f, -1.00f, -1.00f, //J - 1
        1.00f, 0.00f, 3.08f, 0.10f, 0.625f, -1.00f, -1.00f, -1.00f, //A - 2
        0.00f, 4.00f, 0.00f, 0.50f, 1.10f, -1.00f, -1.00f, -1.00f, //K - 3
        //Cara 6
        -1.00f, -0.50f, 3.08f, 0.85f, 0.375f, 1.00f, 1.00f, -1.00f, //B - 4
        -2.62f, 0.00f, 1.90f, 0.45f, 0.46f, 1.00f, 1.00f, -1.00f, //C - 5
        -3.24f, -0.50f, 0.00f, 0.05f, 0.375f, 1.00f, 1.00f, -1.00f, //D - 6
        0.00f, -4.50f, 0.00f, 0.45f, -0.10f, 1.00f, 1.00f, -1.00f, //L - 7
    };
    Mesh* dado = new Mesh();
    dado->CreateMesh(cubo_vertices, cubo_indices, 64, 12);
    meshList.push_back(dado);}

```

Fig. 7. Ajuste de texturas Perico y Cerdo para las caras del dado.

Las demás declaraciones para el dado dentro del main fueron iguales a las de la practica 6, para mandara a llamar a las funciones de cada cara y la localización de las texturas en el proyecto

```
CreateObjects();
Caras_Elef(); Caras_Pulpo(); Caras_Mariq(); Caras_Vaca(); Caras_PC();
CreateShaders();
```

Fig. 8. Declaración de funciones en main.

```
camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 0.5f);

brickTexture = Texture("Textures/brick.png"); dirtTexture.LoadTextureA();
dirtTexture = Texture("Textures/dirt.png"); dirtTexture.LoadTextureA();
plainTexture = Texture("Textures/plain.png"); plainTexture.LoadTextureA();
pisoTexture = Texture("Textures/piso.tga"); pisoTexture.LoadTextureA();
AgaveTexture = Texture("Textures/Agave.tga"); AgaveTexture.LoadTextureA();
elefanteTexture = Texture("Textures/cara_ejefante.tga"); elefanteTexture.LoadTextureA();
pulpoTexture = Texture("Textures/cara_pulpo.tga"); pulpoTexture.LoadTextureA();
mariquitaTexture = Texture("Textures/cara_mariquita.tga"); mariquitaTexture.LoadTextureA();
vacaTexture = Texture("Textures/cara_vaca.tga"); vacaTexture.LoadTextureA();
perico_cerdoTexture = Texture("Textures/cara_perico-cerdo.tga"); perico_cerdoTexture.LoadTextureA();

Lampara = Model(); Lampara.LoadModel("Models/Lampara.obj");
```

Fig. 9. Declaración de ubicación de las texturas.

Para el spotlight que sigue a la cámara, este definido igual al del código proporcionado, solo que los valores de con, lin y exp fueron modificados atendiendo el error que el profesor comento para no hacerlas luces infinitas, en este caso con la nueva configuración se opaca un poco mas con la distancia, para poder observar mejor el reflejo de la luz en cada cara esta debe estar muy cerca del dado.

```
unsigned int spotLightCount = 0;
spotLights[0] = SpotLight(
    1.0f, 1.0f, 1.0f,
    0.0f, 2.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.05f, 0.2f,
    15.0f);
spotLightCount++;
```

Fig. 10. Ajuste de spotlight que sigue a la camara.

```
glm::vec3 lowerLight = camera.getCameraPosition();
lowerLight.y -= 0.3f;
spotLights[0].SetFlash(lowerLight, camera.getCameraDirection());
shaderList[0].SetDirectionalLight(&mainLight);

shaderList[0].SetSpotLights(spotLights, spotLightCount);
```

Fig. 11. Declaración de spotlight que sigue a la camara.

Por ultimo se manda a ejecución las caras con su respectiva textura, esta parte es la misma que la practica 6, solo que en esta ocasión aumente un poco mas la altura de esta con respecto al piso y la coloque en el origen de X y Z.

```
//Ejercicio 1: Agregar dado de 10 caras y que las caras sean iluminadas correctamente.
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 6.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
elefanteTexture.UseTexture(); meshList[4]->RenderMesh();
pulpoTexture.UseTexture(); meshList[5]->RenderMesh();
mariquitaTexture.UseTexture(); meshList[6]->RenderMesh();
vacaTexture.UseTexture(); meshList[7]->RenderMesh();
perico_cerdoTexture.UseTexture(); meshList[8]->RenderMesh();
```

Fig. 12. Declaración de dado a partir de las caras.

- 2) Apagar con teclado la luz (pointlight) de su lámpara creada para el reporte de la práctica 7.

Para esta segunda actividad, arregle el error que comentó el profesor con la asignación de la posición de esta luz puntual, definiéndola desde un inicio en el origen.

```
unsigned int pointLightCount = 0; //Luz de Lampara
pointLights[0] = PointLight(
    1.0f, 1.0f, 1.0f,
    1.8f, 1.0f,
    0.0f, 0.0f, 0.0f,
    1.0f, 0.4f, 0.2f);
pointLightCount++;
```

Fig. 13. Ajuste de luz puntual de la lampara.

Considerando los ajustes de posición que debía darle a la luz a partir de la posición del modelo de la lampara, genere una nueva función en los archivos *PointLight.h* y *PointLight.cpp*, para asígnele esta ubicación

```
void SetPosition(GLfloat x, GLfloat y, GLfloat z);
~PointLight();
```

Fig. 14. Declaración de función para posición en *PointLight.h*.

```
void PointLight::SetPosition(GLfloat x, GLfloat y, GLfloat z)
{
    position.x = x;
    position.y = y;
    position.z = z;
}
```

Fig. 15. Declaración de función para posición en *PointLight.cpp*.

Para ajustar el encendido y apagado de la luz cree una variable booleana para enviar o no la información al shader, definida en el archivo *Window.h*, tanto pública como privada.

```
bool getLampLightState() { return lampLightOn; }
~Window();
```

Fig. 16. Declaración de booleano para encender y apagar luz puntual *Window.h*, publico.

```
static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);

bool lampLightOn;
};
```

Fig. 17. Declaración de booleano para encender y apagar luz puntual *Window.h*, privado.

En el archivo *Window.cpp* declare la variable como booleano true, y después asigno la tecla L para el cambio de arreglos durante la ejecución, se invierte el valor de *lampLightOn*, true a false y viceversa, lo que hará que el código en la siguiente iteración no se mande a shader los valores del spotlight.

```

~Window::Window(GLint windowHeight, GLint windowHeight){
    width = windowHeight;
    height = windowHeight;
    lampLightOn = true;
    for (size_t i = 0; i < 1024; i++){keys[i] = 0;}
}

```

Fig. 18. Declaración de booleano para encender y apagar luz puntual Window.cpp.

```

~void Window::ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode) {
    Window* theWindow = static_cast<Window*>(glfwGetWindowUserPointer(window));

    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
        {glfwSetWindowShouldClose(window, GL_TRUE);}

    if (key == GLFW_KEY_L && action == GLFW_PRESS){
        theWindow->lampLightOn = !theWindow->lampLightOn;

        if (key >= 0 && key < 1024){
            if (action == GLFW_PRESS){theWindow->keys[key] = true;
                //printf("se presiono la tecla %d'\n", key);
            }
            else if (action == GLFW_RELEASE){theWindow->keys[key] = false;
                //printf("se solto la tecla %d'\n", key);
            }
        }
    }
}

```

Fig. 19. Declaración de Letra en el teclado para el encendido y apagado.

En el archivo *main* copie la misma estructura para mandar a ejecutar el modelo de lámpara como el practica 6, pero agregando una jerarquía, *modelaux*, para guardar la ubicación de la a la que se colocó el modelo, después mando a colocar la luz puntual en la ubicación que guarde en *modelaux* pero le agrego unos recorrido extras en X y Y para ubicarlo en el foco del modelo. Por último declaro el If else que mandara al shader la información o no de acuerdo si la tecla L es presionada.

```

//Ejercicio 2: Apagar con teclado la luz (pointlight) de su lámpara.
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.95f, -10.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.05f, 0.05f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara.RenderModel();
glm::vec3 lamparaPos = glm::vec3(modelaux[3]);
pointLights[0].SetPosition(lamparaPos.x + 1.45f, lamparaPos.y + 2.62f, lamparaPos.z);
if (mainWindow.getLampLightState()){shaderList[0].SetPointLights(pointLights, pointLightCount);}
else{shaderList[0].SetPointLights(pointLights, 0);} // Luz apagada, enviamos 0 luces puntuales

```

Fig. 20. Declaración de la luz puntual.

Con todas esas modificaciones, la ejecución del ejercicio fue la siguiente:



Fig. 21. Ejecución del ejercicio.



Fig. 22. Ejecución del ejercicio.

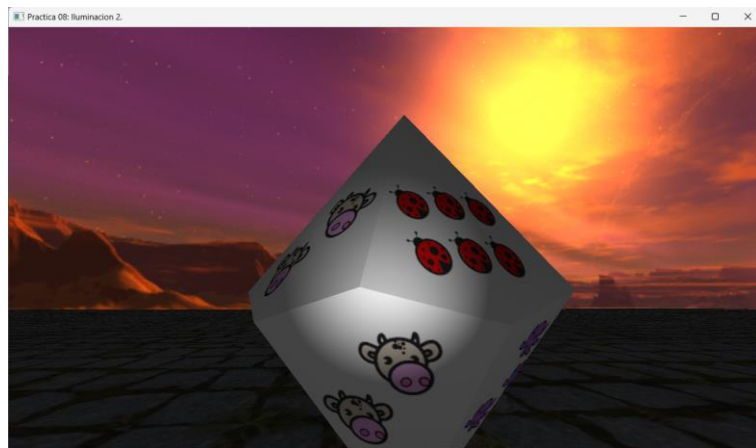


Fig. 23. Ejecución del ejercicio.

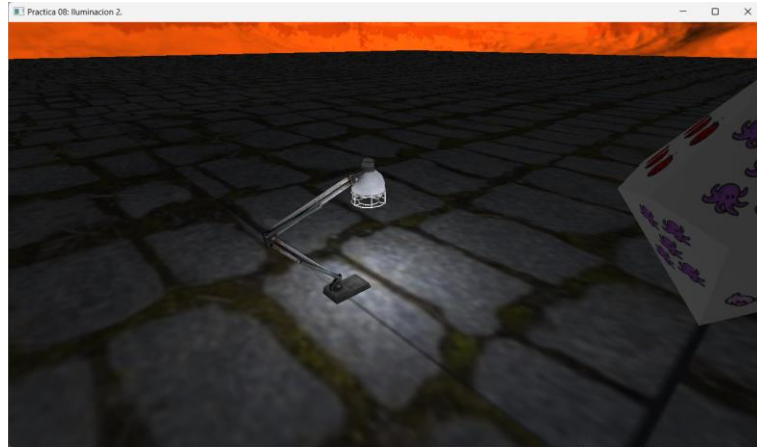


Fig. 24. Ejecución del ejercicio.

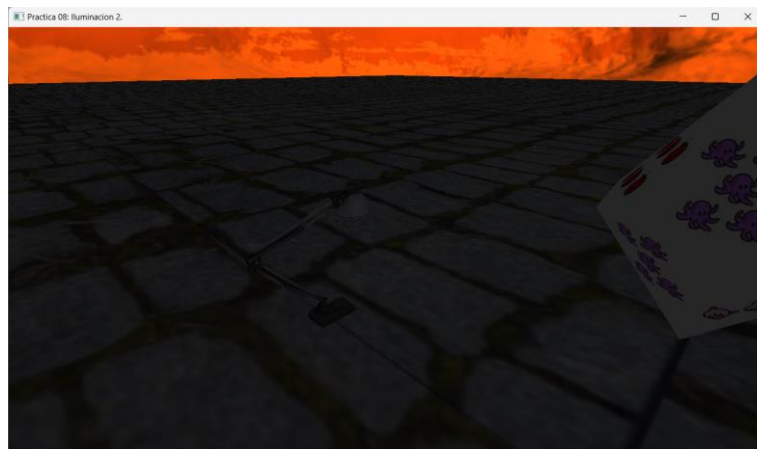


Fig. 25. Ejecución del ejercicio.

2. Problemas presentados.

Para esta actividad no tuve ningún problema, estas actividades fueron un complicadas por las correcciones que debía realizar con respecto a las actividades previas, pero con las aclaraciones del profesor en cada una, las correcciones fueron sencillas, en cuanto a las actividades propias del ejercicio, considero que fueron sencillos, no tuve ningún otro problema.

3. Conclusión:

Tras realizar los ejercicios de la practica me parece que la complejidad era normal, la configuración de las caras del dado para que la luz se reflejara en cada una de acuerdo con el eje en el que debía reflejarse me pareció muy interesante como forma de controlar la luz en objetos creados por el código y no solo modelos, y en el caso de encendido y apagado del foco la actividad no fue tan complicada ya que era algo similar a lo investigado en el previo para cambiar de luces. Al final consideró que esta actividad fue un buen complemento para implementar la iluminación en caras específicas de objetos y para controlar la iluminación de estos, siendo esto a mi parecer una buena práctica para el desarrollo del proyecto final.